

PRODUCTION DOCUMENT INDEX — EVERYTHING A 3D
IMMERSIVE GAME NEEDS

AUDITED AGAINST THE
CODEBASE · AUG 2026

The full document stack a Godot or Unreal production needs, mapped to what already exists in the repo's `design/` and `reference/` folders and in this project. Much is already written — the honest gaps are production crafts, not paper. Status:

EXISTS

THIS PROJECT

TO WRITE

A · DESIGN

DOCUMENT	STATUS	WHERE / WHAT'S NEEDED
Game design document	EXISTS	<code>design/GDD.md</code> — pillars (hijack loop, 4 vehicle classes, 4 factions, grim low-fi aesthetic), pilot-as-persistent-entity architecture, component damage model, UE5-vs-Godot analysis
Game bible (canon)	EXISTS	<code>reference/GAME_BIBLE.md</code> — faction roster, vehicle stats and weapon systems recovered from the original disc's LABTEXT.DAT
Campaign outline	EXISTS	<code>design/CAMPAIGN.md</code> — 20 missions / 4 campaigns, one-new-idea-per-mission rule; needs reconciling with the two-worlds (Moon + Mars) resource-war frame
Vehicle variety spec	EXISTS	<code>design/HAWC_VARIETY_SPEC.md</code> + <code>design/HAWK_rifle_brief.md</code>
Level / encounter design docs	TO WRITE	Per-mission layouts on the HiRISE terrain: spawn logic, cover analysis, wave scripts. One doc per mission, starting with the vertical-slice mission
Systems balance sheet	TO WRITE	Damage/armor/heat numbers as data (CSV or Godot resource), not prose — the disc stats in the game bible are the seed

B · ART & AUDIO

DOCUMENT	STATUS	WHERE / WHAT'S NEEDED
Asset build list	EXISTS	design/ASSET_BUILD_LIST.md – full audit with states. Headline: the 243 MB warrior.glb serves as both hero and enemies; five archetype bases + faction tints replace the whole borrowed roster
Art bible / style guide	THIS PROJECT	Brand sheet + atomic system here carry the 2D language; a 3D art bible (materials, weathering, silhouette rules per class, NASA-photo realism bar) is the missing half
UX / UI spec	THIS PROJECT	Atomic system sheet (atoms → HUD organism). To extend: diegetic cockpit instrumentation per the GDD, menus, globe screen
Audio design doc	TO WRITE	Original soundtrack + SFX already extracted (audio/) – the doc maps them to events and defines the procedural layer
Asset pipeline spec	THIS PROJECT	Spec doc §5 – Blender via MCP, GLB export, naming, poly budgets. Matches the build list's one-Blender-call rule

C · TECHNICAL & PRODUCTION

DOCUMENT	STATUS	WHERE / WHAT'S NEEDED
Technical design doc	EXISTS	GDD §4 – engine strategy (prototype in Godot 4, port the proven slice to UE5 for production fidelity), pilot state machine, component damage tree
Skills & staffing reality	EXISTS	design/SKILLS_REQUIRED.md – the 12-discipline stack; names game feel and art/animation as the hard gaps, scope as the #1 killer
Production plan / milestones	TO WRITE	Vertical slice = the hijack-tutorial mission (it exercises the whole loop). Then: hero mech rebuild → archetype roster → mission variety. Dates against the deck's roadmap
QA / test plan	TO WRITE	The spec's standing 15-minute script (menu → drop → steal → hold wave 3) formalized per build, plus perf gates (60 fps @ 1080p)
Marketing & community plan	TO WRITE	Skills doc says budget 25–40% of effort here. Devlog cadence, trailer beats (orbital drop + one GASHR steal), itch → Steam sequence

DOCUMENT	STATUS	WHERE / WHAT'S NEEDED
Legal / licensing register	EXISTS	CREDITS.md + LICENSE (MIT). Rule from the build list: borrowed CC-BY assets are placeholders to replace with original models

D · THE IMMERSION BAR — AND THE HONEST READ

WHAT "GTA-REAL" MEANS HERE

Not an open-world crime game — the qualities that make one feel real: seamless vehicle↔foot transitions with inherited momentum, diegetic cockpit UI instead of screen chrome, a world that reacts (ejected pilots fight back and reclaim machines, bases are destructible, day/night changes the fight). All three are already specified in the GDD's modern enhancement layer.

THE SCOPE WARNING

The skills doc's own conclusion: scope kills more projects than skill. The paper stack is nearly complete; the gaps are game feel and art/animation — production crafts. The path it recommends stands: make one mech feel great on an empty plane, prove the hijack mission as a vertical slice, then scale. The 20-mission campaign is the destination, not the next step.

E · MCP TOOL STACK — THE AGENT-DRIVEN PIPELINE

BLENDER MCP

Claude operates Blender directly over the MCP connector — modeling, materials and GLB export driven by prompts against the asset build list.

- Skills: hard-surface blockout vocabulary (booleans, bevels, mirrors), PBR node basics, scale discipline (1 unit = 1 m)
- Rule from the build list: build + export in ONE Blender call — the connection is unstable; reapply materials via Godot shader
- Per-asset flow: reference → blockout → detail pass → apply transforms → GLB into `godot/assets/`
- Verify against NASA Mars photos via the `godot-art-pass` loop before accepting

VS CODE CLI · CLAUDE CODE

The code side of the same pipeline: Claude Code in the VS Code terminal works the Godot project — GDScript, scenes, and the Python tools.

- Run Godot headless for import/build checks: `godot --headless --path godot --quit` after every asset drop
- Drive the existing `tools/*.py` (heightmaps, normalizers, region extractors) as CLI steps, not manual runs
- Project skills live in `.claude/skills/` in the repo — add one skill per recurring job (art pass, asset import, mission scaffold)
- Git + LFS hygiene per change; CI nightly export as the end state

Together they close the loop the skills doc worries about: Blender MCP covers the art-production gap asset by asset, the CLI keeps every result verified in-engine the same hour it's made.